Johanna Fan

CPSC 482

Spring 2026

Final Report

Project Overview and Goals

For this project I built an end-to-end data analytics pipeline using publicly available Airbnb listing data from Inside Airbnb. The goal was to design a real cloud-based data engineering workflow from scratch by ingesting raw CSV files, cleaning and transforming them in the cloud, training a machine learning model to predict nightly prices, and presenting insights through an interactive dashboard.

My specific goals were to:

- Combine Airbnb listing data across 15 U.S. cities into one unified dataset
- Store raw data in Google Cloud Storage and process it entirely within BigQuery
- Build a BigQuery ML linear regression model to predict nightly listing price
- Visualize key pricing patterns through a Looker Studio dashboard

The motivation was that Airbnb pricing is a genuinely messy real-world problem. Prices vary a lot across cities, neighborhoods, and listing types, and I wanted to understand what actually drives those differences at a data level rather than just guessing.

Dataset Description

Data comes from Inside Airbnb http://insideairbnb.com/get-the-data/, which publishes scraped Airbnb listing snapshots for cities worldwide. I ingested CSV files for 15 U.S. cities: Austin, Boston, Chicago, Dallas, Denver, Hawaii, Los Angeles, Nashville, New Orleans, New York City, Portland, San Diego, Seattle, San Francisco, and Washington DC. After cleaning, 9 cities had usable price data and made it into the final analysis: Austin, Chicago, Denver, Hawaii, Nashville, New Orleans, San Diego, Seattle, and Washington DC. The remaining 6 cities (NYC, LA, SF, Boston, Portland, Dallas) were excluded because their price columns loaded as NULL.

The key columns used throughout the pipeline:

Column	Description
city	City label (added during ingestion)
neighbourhood_cleansed	Neighborhood name
room_type	Entire home/apt, Private room, Hotel room, Shared room
price	Nightly price
bedrooms	Number of bedrooms
review_scores_rating	Overall review score (0–5)
availability_365	Days available in next 365 days
amenities	Raw text list

Across all 15 cities the raw dataset totaled 613,434 rows. The data was messy, such as prices stored as dollar-sign strings, amenities as unformatted text arrays, missing values throughout, and minor schema differences between city files.

Pipeline Architecture and GCP Services

The pipeline follows a straightforward layered architecture entirely on GCP:

Inside Airbnb CSVs (15 cities)

↓

Google Cloud Storage

↓

BigQuery (staging → cleaned → analysis tables)

↓

BigQuery ML (linear regression)

↓

Looker Studio (dashboard)

Services used:

- Google Cloud Storage: Raw CSV files were uploaded here first as the immutable source layer.
- BigQuery: Main transformation and analytical engine. Used for cleaning, feature engineering, joining city datasets, building analysis tables, and training the ML model.
- BigQuery ML: Linear regression model trained directly inside BigQuery on the cleaned table.
- Looker Studio: Connected directly to BigQuery for the final dashboard.

Data Processing Steps

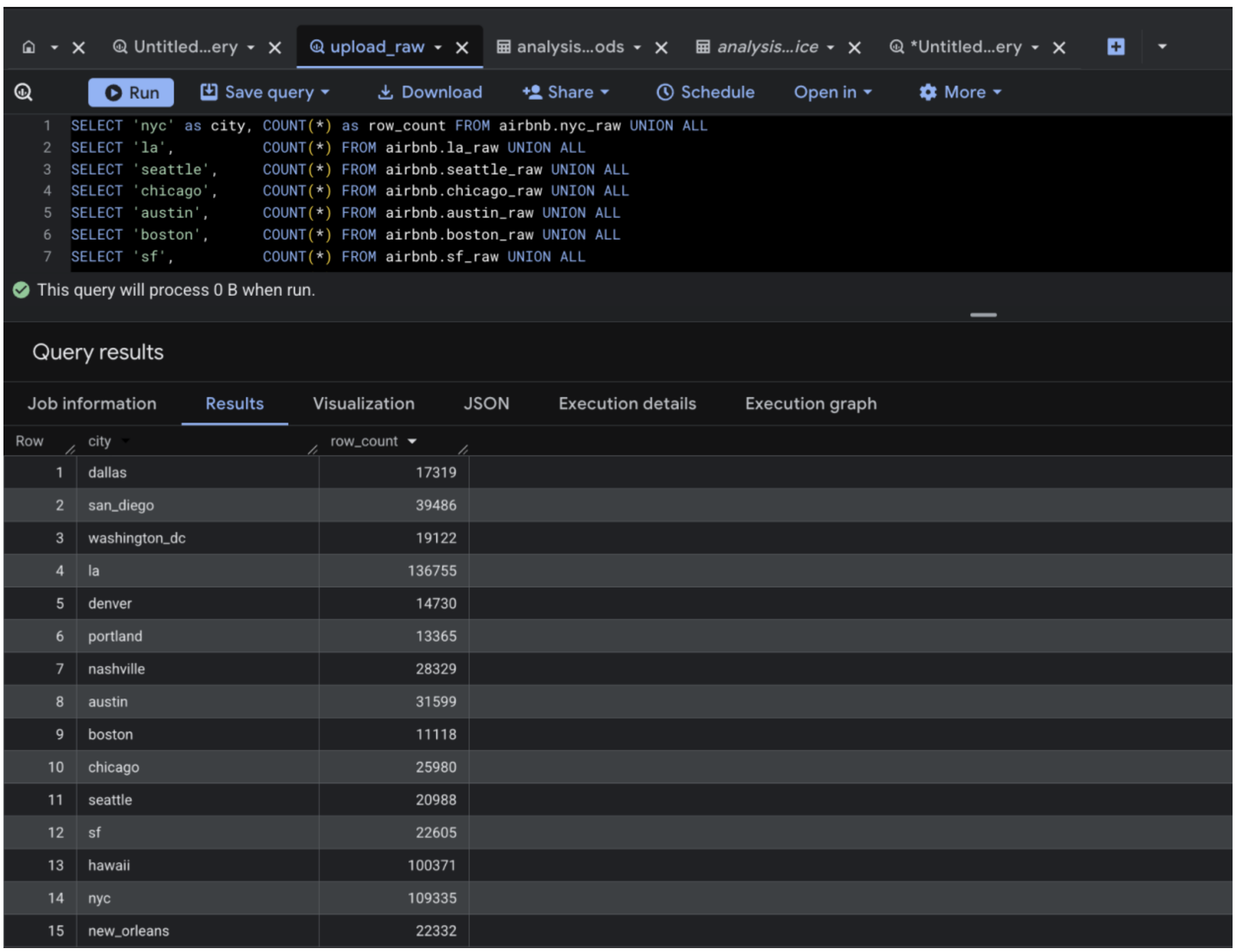
Each city's CSV was loaded from GCS directly into its own raw staging table using BigQuery's `LOAD DATA` statement:

```
LOAD DATA INTO airbnb.nyc_raw
FROM FILES (
  format = 'CSV',
  uris = ['gs://airbnb-pipeline-data/raw/nyc/listings.csv'],
  skip_leading_rows = 1,
```

```
    allow_quoted_newlines = true
);
```

This was repeated for all 15 cities. To verify the ingestion I ran a row count across all raw tables:

```
SELECT 'nyc' as city, COUNT(*) as row_count FROM airbnb.nyc_raw UNION ALL
SELECT 'la',       COUNT(*) FROM airbnb.la_raw UNION ALL
SELECT 'seattle',  COUNT(*) FROM airbnb.seattle_raw UNION ALL
-- ... (all 15 cities)
```



Row	city	row_count
1	dallas	17319
2	san_diego	39486
3	washington_dc	19122
4	la	136755
5	denver	14730
6	portland	13365
7	nashville	28329
8	austin	31599
9	boston	11118
10	chicago	25980
11	seattle	20988
12	sf	22605
13	hawaii	100371
14	nyc	109335
15	new_orleans	22332

Results confirmed all 15 cities loaded successfully with a combined total of 613,434 raw rows.

The main cleaning step unified all 15 city tables, parsed the price column from string to numeric, extracted amenity count from the raw text array, and handled null values. The transformed output was saved as `airbnb.listings_clean`.

Key transformations:

- `price`: stripped `$` and `,` with `REGEXP_REPLACE`, cast to `FLOAT64`, filtered to valid range ($10–$5,000)
- `amenities`: used `REGEXP_REPLACE` + `SPLIT` + `ARRAY_LENGTH` to get amenity count per listing
- `bedrooms` and `review_scores_rating`: null values filled with reasonable defaults
- All 15 city tables unified with `UNION ALL` with a `city` label added per table

After cleaning: 248,496 listings remained across 9 cities, which is a reduction from 613,434 raw rows, driven by 6 cities whose price columns were entirely NULL plus out-of-range and malformed records in the remaining cities.

Five analysis tables were built from `listings_clean` to power the dashboard and explore patterns:

- Price by city

```
CREATE OR REPLACE TABLE airbnb.analysis_price_by_city AS
SELECT city, COUNT(*) AS listings,
  ROUND(AVG(price_usd), 2) AS avg_price,
  ROUND(MIN(price_usd), 2) AS min_price,
  ROUND(MAX(price_usd), 2) AS max_price
FROM airbnb.listings_clean
GROUP BY city ORDER BY avg_price DESC;
```

- Price by room type

```
CREATE OR REPLACE TABLE
airbnb.analysis_price_by_room_type AS
SELECT room_type, COUNT(*) AS listings,
  ROUND(AVG(price_usd), 2) AS avg_price
FROM airbnb.listings_clean
GROUP BY room_type ORDER BY avg_price DESC;
```

- Availability tiers vs price

```
CREATE OR REPLACE TABLE airbnb.analysis_availability AS
SELECT
  CASE
    WHEN availability_365 < 30  THEN 'Rarely Available
(<30 days)'
    WHEN availability_365 < 90  THEN 'Occasional (30-90
days)'
    WHEN availability_365 < 180 THEN 'Moderate (90-180
days)'
    ELSE 'Highly Available (180+ days)'
  END AS availability_tier,
  COUNT(*) AS listings,
  ROUND(AVG(price_usd), 2) AS avg_price
FROM airbnb.listings_clean
GROUP BY availability_tier ORDER BY avg_price DESC;
```

- Review scores vs price

```
CREATE OR REPLACE TABLE
airbnb.analysis_review_vs_price AS
SELECT
  CASE
    WHEN review_score >= 4.8 THEN '4.8-5.0
(Excellent)'
    WHEN review_score >= 4.5 THEN '4.5-4.8 (Great)'
    WHEN review_score >= 4.0 THEN '4.0-4.5 (Good)'
    WHEN review_score > 0    THEN 'Below 4.0'
    ELSE 'No Rating'
  END AS score_bucket,
  COUNT(*) AS listings,
  ROUND(AVG(price_usd), 2) AS avg_price
FROM airbnb.listings_clean
GROUP BY score_bucket ORDER BY avg_price DESC;
```

- Top neighborhoods

```
CREATE OR REPLACE TABLE
airbnb.analysis_top_neighborhoods AS
SELECT city, neighborhood, COUNT(*) AS listings,
  ROUND(AVG(price_usd), 2) AS avg_price
FROM airbnb.listings_clean
GROUP BY city, neighborhood HAVING listings > 20
ORDER BY avg_price DESC LIMIT 50;
```

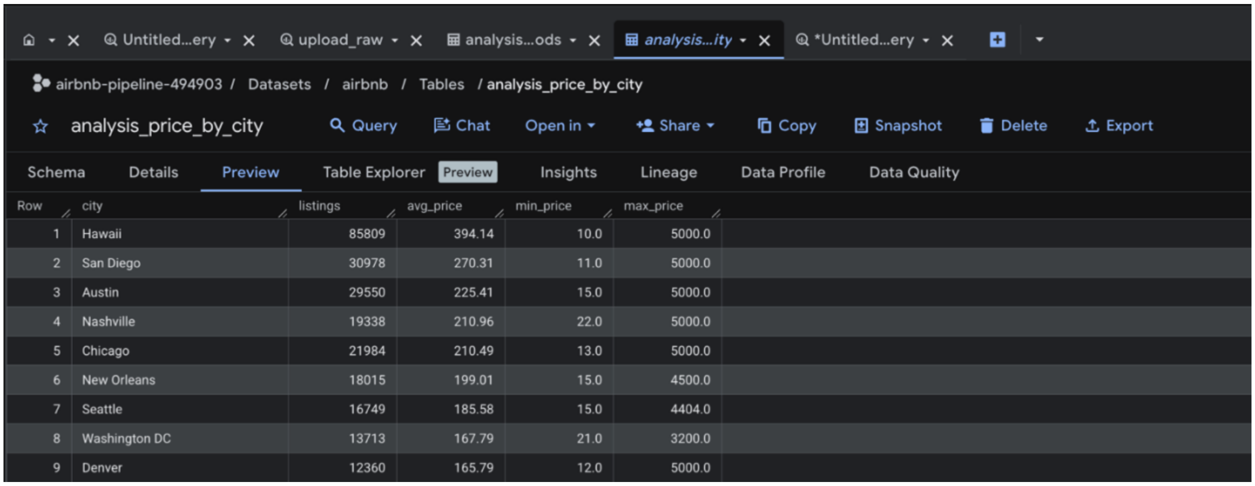
Results

Raw Ingestion

City	Raw Row Count
NYC	109,335
Hawaii	100,371
LA	136,755
Austin	31,599
San Diego	39,486
Nashville	28,329
Chicago	25,980
San Francisco	22,605
New Orleans	22,332
Washington DC	19,122
Seattle	20,988
Dallas	17,319
Denver	14,730
Portland	13,365
Boston	11,118
Total	613,434

Note: All 15 cities loaded successfully into raw BigQuery tables. However, 6 cities (NYC, LA, SF, Boston, Portland, Dallas) had NULL price columns after ingestion and were excluded from the cleaned dataset. The analysis below reflects the 9 cities that had valid price data.

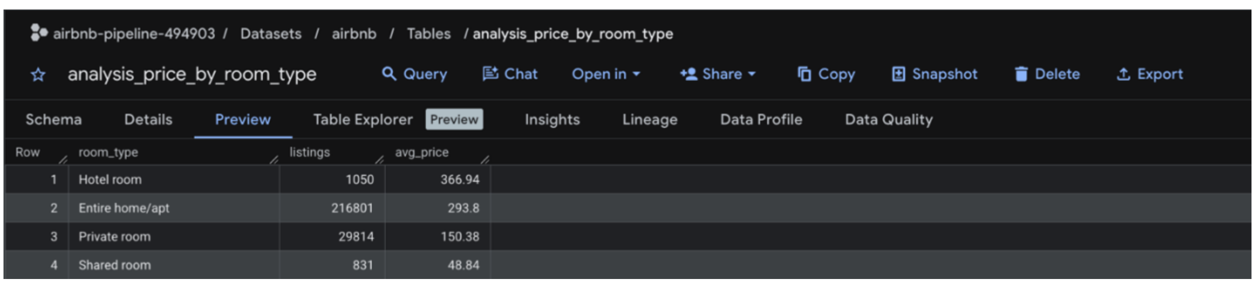
Price by City (cleaned listings)



Row	city	listings	avg_price	min_price	max_price
1	Hawaii	85809	394.14	10.0	5000.0
2	San Diego	30978	270.31	11.0	5000.0
3	Austin	29550	225.41	15.0	5000.0
4	Nashville	19338	210.96	22.0	5000.0
5	Chicago	21984	210.49	13.0	5000.0
6	New Orleans	18015	199.01	15.0	4500.0
7	Seattle	16749	185.58	15.0	4404.0
8	Washington DC	13713	167.79	21.0	3200.0
9	Denver	12360	165.79	12.0	5000.0

Hawaii was the most expensive market by a significant margin at $394.14 average, followed by San Diego ($270.31) and Austin ($225.41). Washington DC ($167.79) and Denver ($165.79) were the most affordable among the cities shown.

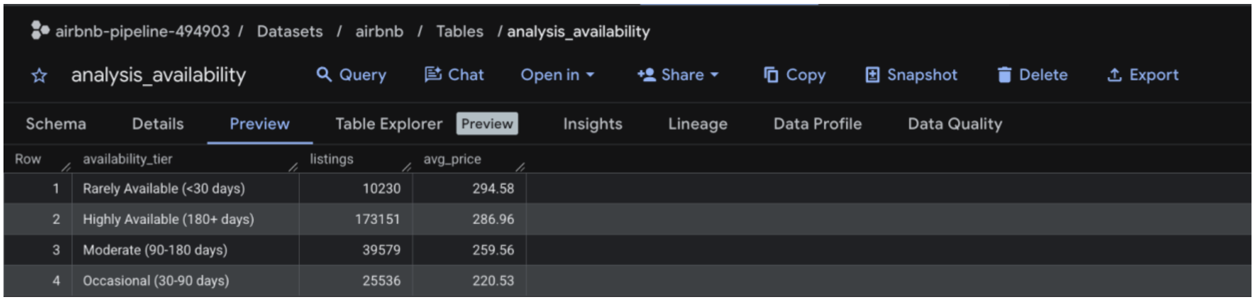
Price by Room Type



Row	room_type	listings	avg_price
1	Hotel room	1050	366.94
2	Entire home/apt	216801	293.8
3	Private room	29814	150.38
4	Shared room	831	48.84

Entire home/apt listings completely dominate the dataset at 216,801 out of 248,496 total (87%). Hotel rooms average the highest price but represent less than 0.5% of listings.

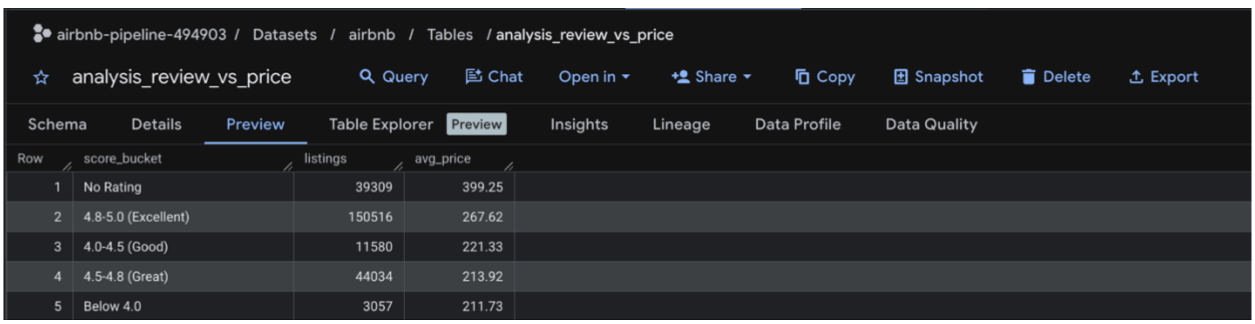
Availability vs. Price



Row	availability_tier	listings	avg_price
1	Rarely Available (<30 days)	10230	294.58
2	Highly Available (180+ days)	173151	286.96
3	Moderate (90-180 days)	39579	259.56
4	Occasional (30-90 days)	25536	220.53

Interestingly, rarely available listings and highly available listings both average higher than mid-tier availability listings. The rarely available group likely includes premium/seasonal rentals that are only listed briefly.

Review Scores vs. Price



Row	score_bucket	listings	avg_price
1	No Rating	39309	399.25
2	4.8-5.0 (Excellent)	150516	267.62
3	4.0-4.5 (Good)	11580	221.33
4	4.5-4.8 (Great)	44034	213.92
5	Below 4.0	3057	211.73

The "No Rating" group has the highest average price ($399.25), which likely reflects newer or luxury listings that haven't accumulated reviews yet. Among rated listings, higher scores correlate weakly with higher price — excellent listings average $267.62 vs $211.73 for below 4.0.

BigQuery ML Model

The model was trained using an 80/20 random train/test split:

```
CREATE OR REPLACE MODEL airbnb.price_model
OPTIONS (
  model_type = 'linear_reg',
  input_label_cols = ['price_usd'],
```

```
  data_split_method = 'RANDOM',
  data_split_eval_fraction = 0.2
) AS
SELECT
  city, neighborhood, room_type, review_score,
  availability_365, bedrooms, amenity_count, price_usd
FROM airbnb.listings_clean
WHERE price_usd IS NOT NULL AND review_score IS NOT NULL;
```

Evaluation results:

```
SELECT
  ROUND(mean_absolute_error, 2) AS MAE,
  ROUND(SQRT(mean_squared_error), 2) AS RMSE,
  ROUND(r2_score, 4) AS R2
FROM ML.EVALUATE(MODEL airbnb.price_model);
```

Metric	Value
MAE	144.54
RMSE	296.19
R^2	0.34

An R^2 of 0.34 means the model explains about 34% of price variance. On average, predictions are off by about $144 (MAE). The high RMSE (296) relative to MAE (144) indicates the model struggles with high-end outlier listings.

Visualizations / Dashboard

The Looker Studio dashboard connects directly to the BigQuery analysis tables and presents the key findings in a single page with a city filter dropdown at the top right to slice all charts by city. The dashboard includes six components:

1. ML Model Metrics (top left scorecard): $R^2 = 0.34$, RMSE = 296.19, MAE = 144.54 — surfaced directly on the dashboard so the model's predictive performance is visible alongside the descriptive analysis.
2. Average Nightly Price by City (bar chart): Shows the 9 cities visible after cleaning, ordered by average price. Hawaii dominates at $394, followed by San Diego ($270) and Austin (~$225). Denver and Washington DC are the most affordable at ~$166 and ~$168 respectively.
3. Listing Type Distribution (pie chart): Entire home/apt listings make up 87.2% of the dataset, private rooms account for 12%, and hotel rooms and shared rooms together represent less than 1%.
4. Average Price by Number of Bedrooms (bar chart): Clear upward trend — 1-bedroom listings average around $175 while 6-bedroom listings average nearly $950. This was one of the stronger linear signals in the dataset.
5. Review Score vs. Average Price (table): Interestingly, unrated listings have the highest average price ($399.25), while rated listings show only a weak relationship between score and price — excellent listings (4.8–5.0) average $267.62 vs. $211.73 for below 4.0.
6. Top 50 Neighborhoods by Price (table): The priciest neighborhoods are Austin zip code 78732 ($906.40, 156 listings), Nashville's District 34 ($818, 36 listings), and Hawaii's Lahaina ($655.40, 13,191 listings). Austin zip codes appear three times in the top six, suggesting concentrated luxury rental activity there.

https://datastudio.google.com/reporting/cfbf3e74-1fb3-42a6-9df7-fbd5af684fa2

Challenges and How I Resolved Them

- Six cities dropped due to NULL price columns: NYC, LA, SF, Boston, Portland, and Dallas all loaded into BigQuery successfully, but their price columns came through entirely as NULL. This was likely caused by a schema difference in those city CSVs, possibly a different column ordering or encoding that caused the price field to not map correctly during the LOAD DATA step. As a result, the final cleaned dataset covers 9 of the 15 originally ingested cities. Given more time, I would debug the LOAD DATA schema mapping for each affected city and re-ingest them.
- Price column parsing: The price field came in as a string. A direct `CAST` to FLOAT64 failed immediately. I had to chain two `REGEXP_REPLACE` calls before the cast would work cleanly.
- Schema differences across 15 cities: Each city's CSV had the same column names but different null representations and column orderings. Some cities used empty strings where others had actual NULLs. I handled this in the cleaning layer with explicit `COALESCE` and `WHERE price IS NOT NULL AND price != ''` filters rather than trying to normalize anything at the GCS level.
- Amenity count extraction: Amenities were stored as a raw string that looked like a Python list: `["Wifi", "Kitchen", "TV"]`. I didn't need individual amenity names, just the count per listing, so I used regex to strip brackets and quotes, split on commas, and called `ARRAY_LENGTH`. It's a bit ugly but consistent across all cities.

- ML model initial confusion: My first model attempt had worse results than expected. I realized I needed to be explicit about filtering out rows where `review_score IS NULL` in the training query. After adding that filter the results improved.

Lessons Learned

The biggest thing I took away from this project is that data cleaning is where almost all the real work lives. I expected to spend most of my time on the ML model or the dashboard, but the cleaning and transformation steps took up the majority of my time. None of it was individually hard, but together there were a lot of edge cases.

I also came to genuinely appreciate why BigQuery was the right tool here. Having the storage, querying, transformation, and ML all inside one system meant I never had to export data between environments. The fact that `ML.EVALUATE` runs right there in the same query interface where I built my tables made iteration much faster than I expected.

The R^2 of 0.34 was honestly a bit humbling. It showed me clearly that the structured attributes I used only capture part of what drives price. The real signal is probably location at a finer granularity (exact coordinates, proximity to transit or landmarks, neighborhood desirability) and that's just not in this dataset.

I also learned that the relationship between review score and price is not what I assumed going in. I expected higher-rated listings to cost more, but the data shows that unrated listings are actually the most expensive on average. That's the kind of counterintuitive finding that only shows up when you actually look at the data rather than assuming.

Potential Next Steps

If I had more time, here's what I would prioritize:

1. Add geographic coordinates as features: Latitude and longitude may improve the model's R^2 significantly. Location at a fine grain is the main driver of price that my current features don't capture.
2. Use a more powerful model: BigQuery ML supports boosted tree models (XGBoost-style). A gradient boosted tree would handle the skewed price distribution and non-linear interactions between features much better than linear regression. I'd expect meaningful R^2 improvement.
3. Expand to seasonal analysis: Inside Airbnb releases new snapshots periodically. Joining multiple time snapshots for the same cities would let me analyze how prices shift seasonally which is more useful for actual hosts or travelers.
4. Automate the pipeline: Everything is currently triggered manually. Setting up Cloud Scheduler to pull fresh data monthly and re-run the full pipeline would turn this into a live, maintained system rather than a one-time analysis project.
5. Deeper neighborhood analysis: The `analysis_top_neighborhoods` table only surfaces the top 50 by average price. A more complete neighborhood-level analysis, which would include price per bedroom, review score distribution, and availability patterns by neighborhood, and it would make the dashboard significantly more useful.